

Assignment Report

Soham Pandit

IIT Indore

1 Introduction

1.1 Plan of Action

I follow the lifecycle of AI development mentioned in this whitepaper[3]. Although it is for Gen AI system development traditional ML systems follow the same lifecycle in essence.

2 Exploratory Data Analysis & Feature Engineering

All the data was fetched from the database and stored in CSV files. First of all I checked for class imbalance in training data which was found to be mild with False(66%) and True(34%). Next I dived into the masked features of the user in the features table. Please note that a deeper EDA and justified feature engineering choices can be found in the Jupyter Notebooks.

2.1 Features Table

Many columns had high missingness with feature_8 having the highest of around 72%. Such high values of missingness meant I could not use imputation methods. Rather I tried to model the missingness as a feature so I did the following: created a binary column marking NaN and numeric values as categories. Next to understand if they affect the label I built a Hypothesis Testing scenario and used a Chi-Square Test because we had sample distribution with categorical values.

Null Hypothesis (H_0): Missingness in any masked feature is independent of the target outcome(is.repaid).

Alternate Hypothesis (H_1): Missingness in any masked feature is dependent of the target outcome(is.repaid).

feature_3's binary missing column was a significant feature because p-value was less than 0.05, thwarting the Null Hypothesis. After that did simple visualization of the plots to check for non-linearity and categorical features using box plots and kernel density estimates.

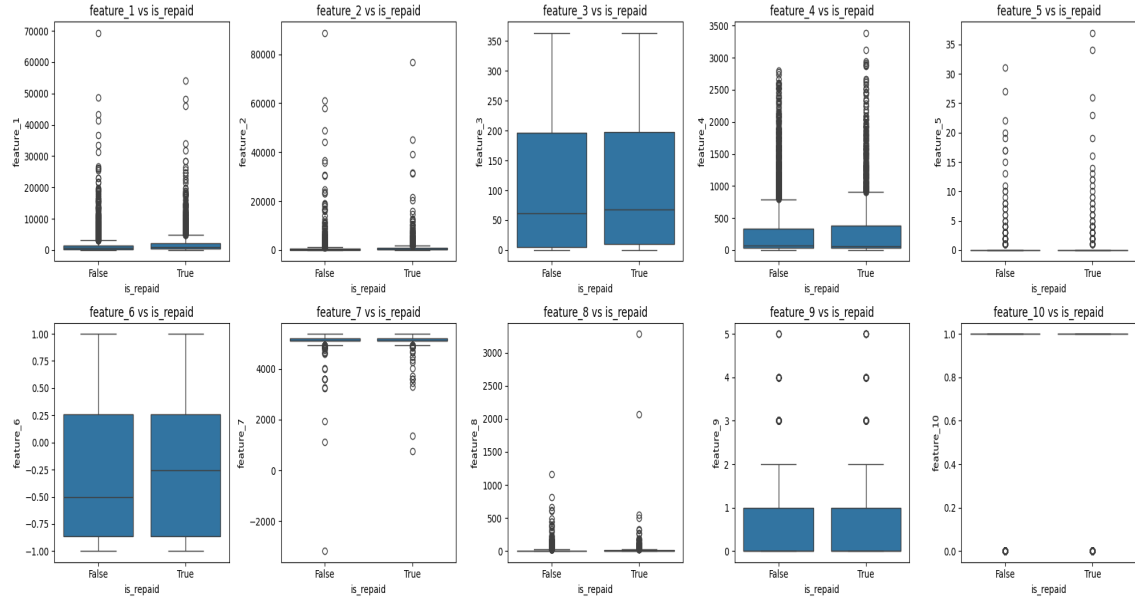


Figure 1: Box plots of the masked features.

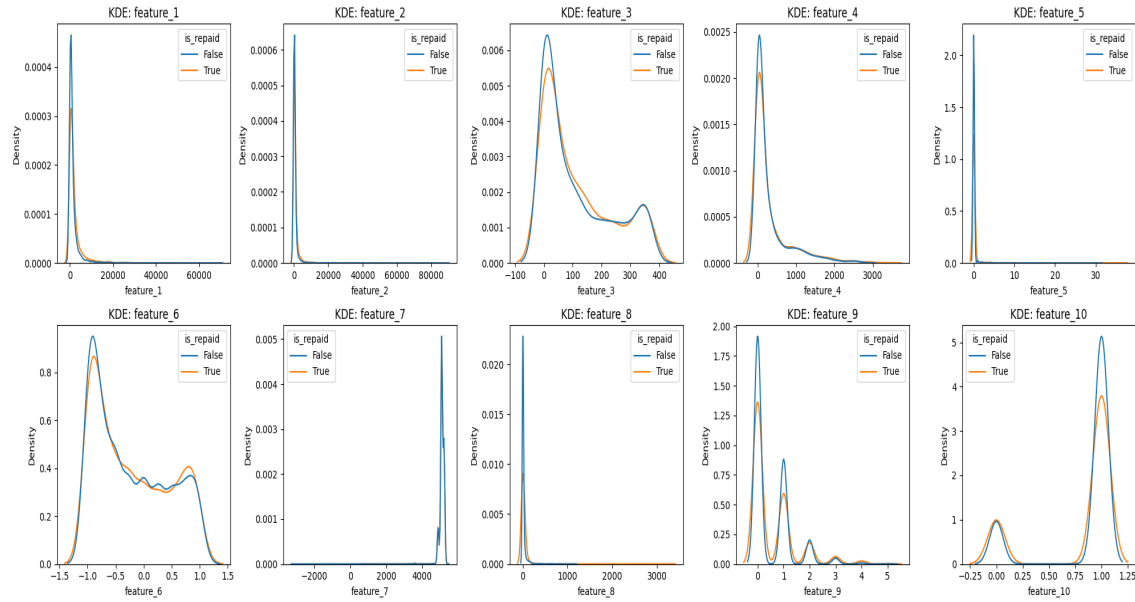


Figure 2: Kernel Density Estimates of the masked features.

With non-linearity confirmed from the plots mostly via visualization I chose Mutual Information to rank the features importance-wise to the labels. I still used Spearman correlation for sake of categorical labels and underlying linearity.

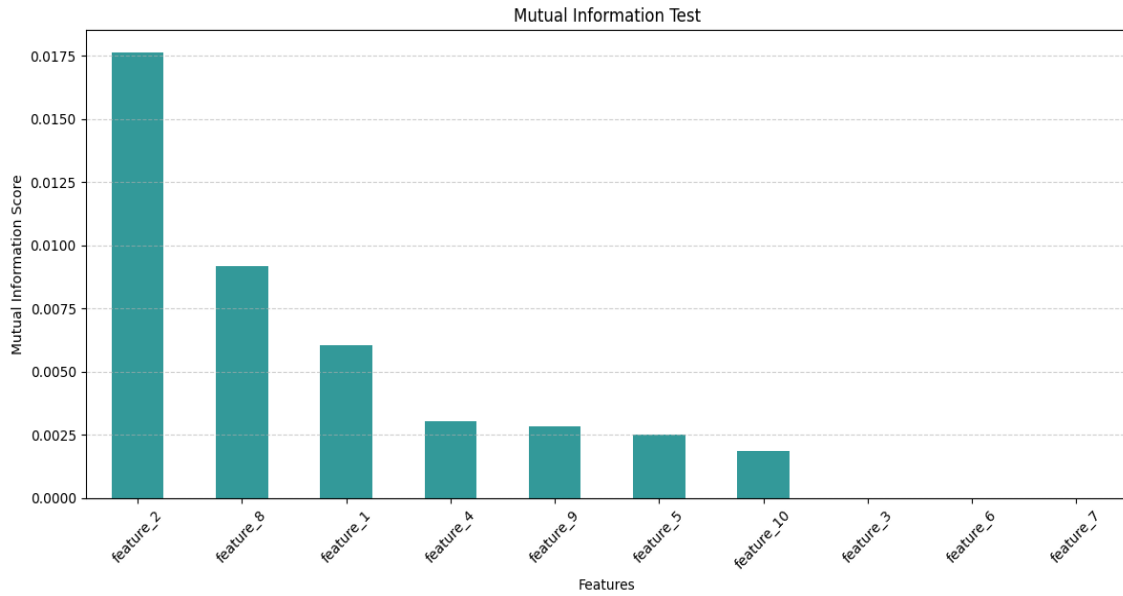


Figure 3: Features ranked by Mutual Information.

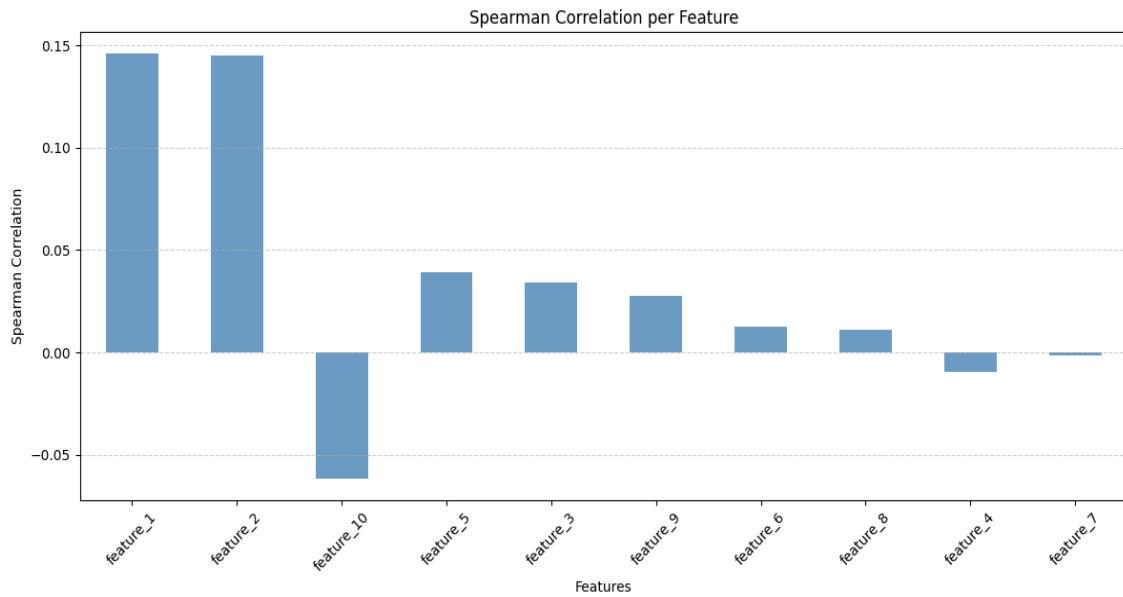


Figure 4: Features ranked by Spearman Correlation.

2.2 Events Table

Events table gave a great deal of information about the behavior of the users. I had derived many features from the data present in the table but have included only the ones which improved performance.

- **num_sessions_rank**: Captures how frequently a user engages with the app relative to others, serving as a proxy for overall user engagement and intent.
- **active_days_rank**: Measures how consistently a user returns to the app across days, reflecting behavioral regularity rather than bursty or one-off usage.
- **session_duration_rank**: Represents the relative depth of user engagement by ranking how long users spend per session, which can indicate attention and seriousness.
- **recency_rank**: Encodes how recently a user interacted with the app before applying, capturing short-term behavioral momentum or dormancy.
- **diligence_stability**: Combines engagement depth with location stability to capture disciplined, repeat usage patterns rather than erratic behavior.
- **mean_bigram_default_rate**: Estimates average default risk based on common screen-to-screen navigation patterns, capturing typical behavioral flows associated with repayment outcomes.
- **max_bigram_default_rate**: Highlights the highest-risk navigation pattern observed for a user, acting as a stress signal for potentially risky behavioral sequences.

Behavioral features were designed to capture how users interact with the application leading up to a loan request, with an emphasis on consistency, recency, and depth of engagement rather than raw activity volume. Rank-based representations were used to normalize behavior across users and reduce sensitivity to outliers, while interaction features and sequence-based signals were introduced to capture higher-order patterns such as disciplined usage and risky navigation flows. Together, these features reflect meaningful user intent and stability, providing strong predictive signal without relying on brittle thresholds or single-event behaviors.

2.3 GPS Table

Similar to events table I am including only the important features here:

- **max_land_speed**: Captures the highest observed movement speed before application, acting as a coarse indicator of mobility intensity and device usage context.
- **gps_points_pre_app**: Measures the volume of location signals recorded prior to application, serving as a proxy for app usage frequency and sensor availability.
- **gps_points_pre_app_log**: Log-scaled version of GPS activity to reduce the influence of extreme usage while preserving relative differences between users.
- **unique_locations**: Counts distinct geographic locations visited, reflecting the breadth of a user's movement rather than raw activity volume.

- **dominant_location_ratio**: Quantifies how concentrated a user’s location history is around a single place, capturing routine stability versus dispersed movement.
- **location_entropy**: Measures the unpredictability of a user’s movement patterns, with higher values indicating more varied and less regular location behavior.
- **radius_of_gyration**: Represents the spatial spread of a user’s movement around their center of activity, summarizing overall mobility range.
- **radius_of_gyration_log**: Log-scaled mobility spread that dampens outliers while preserving differences between low- and high-mobility users.

GPS features were included to capture mobility patterns and location stability as complementary signals to app engagement. Rather than modeling precise trajectories, the features summarize movement concentration, spatial spread, and variability, allowing the model to distinguish routine behavior from irregular or highly dispersed movement. Log transformations were applied to mitigate the impact of extreme values, and the overall feature set was designed to provide incremental context while avoiding over-reliance on noisy or sparse location data.

Most of the activity was spread in the southern region of Kenya around the city of Nairobi and the eastern coast of Kenya.

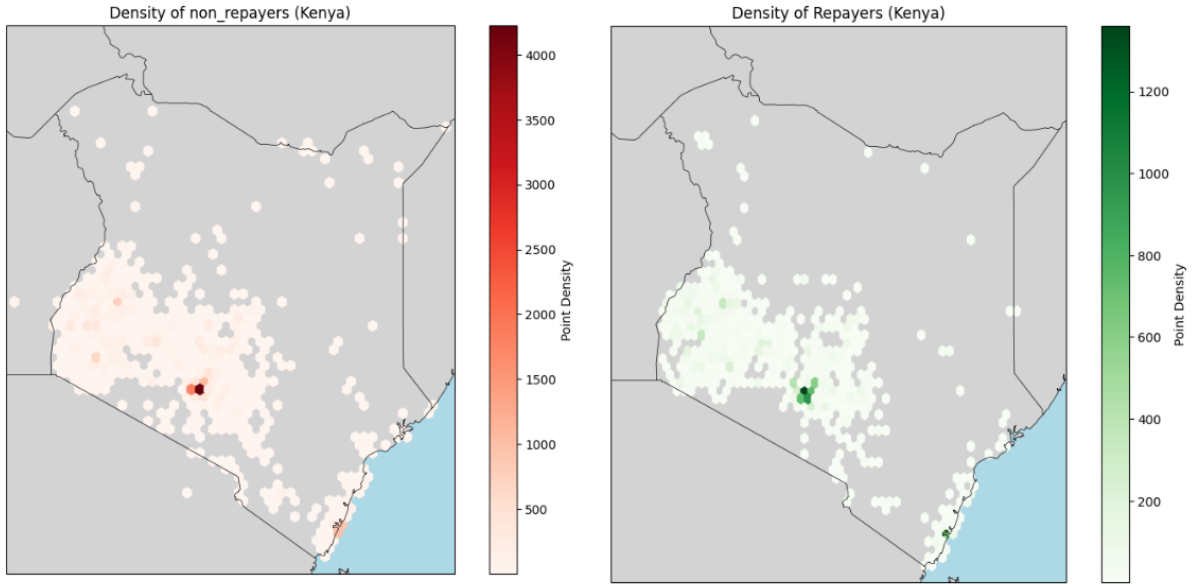


Figure 5: Repayers and Non-Repayers are highest in Kenya than anywhere in the world..

2.4 Temporal Features

Temporal features provided contextual adjustments to the model by capturing when users apply, rather than how they behave, allowing the model to account for timing-related effects

without dominating predictions.

- **app_hour**: Captures the time of day when the application was submitted, reflecting short-term contextual and behavioral timing patterns.
- **app_day_of_week**: Encodes weekly application timing, capturing differences between weekday and weekend user behavior.
- **app_day_of_month**: Represents monthly timing effects, such as behavior around pay cycles or recurring financial obligations.
- **app_year**: Accounts for long-term temporal effects, including changes in user behavior or macro conditions across years.
- **is_weekend**: Flags applications submitted on weekends, when user intent and financial behavior may differ from weekdays.
- **is_late_night**: Identifies applications submitted during late-night hours, capturing potential urgency or atypical usage patterns.

3 Model Choice

Sticking to tabular data and the time constraints I decided onto using tree-based bagging and boosting models due to the following reasons:

- **Ability to capture nonlinear behavior**: Borrower risk depends on complex, nonlinear relationships between engagement, mobility, and timing features, which tree-based models naturally capture without extensive feature transformations. [4]
- **Robustness to mixed feature types**: The dataset contains a mix of continuous, ordinal, binary, and rank-based features, along with missing values, all of which tree-based models handle effectively without heavy preprocessing.
- **Graceful handling of missingness**: Tree-based models can learn informative splits on missing values or missingness indicators, making them well-suited for real-world mobile data where sensor availability is inconsistent. [1]
- **Strong performance on tabular data**: Gradient-boosted trees such as XGBoost, CatBoost, and LightGBM are well-established baselines for tabular problems and consistently outperform linear and neural approaches in similar credit-risk settings.[2]
- **Interpretability and explainability**: Tree-based models integrate seamlessly with SHAP, allowing for clear, consistent feature attribution and supporting transparency requirements common in lending and risk applications.

- **Stability under limited data:** With a moderate dataset size, tree-based ensembles provide strong generalization without the data hunger or tuning complexity associated with deep learning models.

Given the structure of the data and the need for both performance and explainability, tree-based models offered the best balance between predictive power, robustness, and interpretability.

Apart from this stacking with boosting models was tried to increase performance but it was marginal and needs more work.

4 Experimental Setting

The dataset was split into a 90/10 training and validation sets for evaluation. The tabular data had 28 columns ranging from user, behavioral, spatial and temporal features. The dataset was sorted chronologically and then split to prevent data leakage, since the model will not have access to the trends or patterns observed in the future. Similar strategies were used to model the features, and only those instances which occurred before the user applied were considered.

5 Results

5.1 Choice of Metrics

The models were evaluated on the following 4 metrics:

- **ROC-AUC:** The probability that a randomly chosen positive instance is assigned a higher score than a randomly chosen negative instance.
- **PR-AUC:** The area under the precision–recall curve, summarizing the trade-off between precision and recall across classification thresholds.
- **Log Loss:** A strictly proper scoring rule that measures the negative log-likelihood of the true labels given the predicted probabilities.
- **Brier Score:** The mean squared error between predicted probabilities and the observed binary outcomes, measuring probability calibration.

Rather than focusing on the mathematical aspects of the metrics I want to focus on what the metrics helped me answer and how did they create a more comprehensive view for the model evaluation. AUC-ROC clarifies that "If I randomly pick one borrower who defaulted and one who repaid, how often does the model rank the defaulter as riskier?" checking whether the model can correctly rank risky borrowers above safe ones, regardless of where the decision threshold is set. PR-AUC answers the question "When the model

Model Names	Metrics			
	AUC-ROC	PR-AUC	Log Loss	Brier Loss
Decision Trees	0.5296	0.3555	0.6705	0.2389
Random Forest	0.5668	0.3940	0.6177	0.2127
CatBoost	0.5999	0.4361	0.6456	0.2265
LightGBM	0.5829	0.4283	0.6407	0.2248
XGBoost	0.6008	0.4231	0.6065	0.2083

Table 1: Performance of tree-based models on the validation set.

flags someone as risky, how often is it actually correct?”, obviously similar to to common classification metrics of Precision and Recall. Log Loss answer the following: ”How good are the predicted probabilities themselves?” penalizing the model when it confidently answers wrong. Brier Loss/Score answer the question of calibration ”Do the model’s predicted probabilities match real-world outcomes?” and checks if the model probabilities are consistent, rather than just ordered correctly.

It is important to note that I did not use metrics like Accuracy, Precision, Recall and F1 score since probabilities were required so there was no thresholding to classify the data into the binary classes.

The decision tree model shows limited usefulness for lending decisions, as reflected by its low ROC-AUC and PR-AUC, which indicate weak risk ranking and poor identification of likely defaulters. Its higher log loss and Brier score further suggest unreliable probability estimates, making it unsuitable for decisions that depend on accurate risk scoring. The random forest improves meaningfully on these metrics, but it still falls short of providing consistently strong confidence in borrower risk.

Gradient-boosted models perform best overall, demonstrating stronger risk ranking and default detection across ROC-AUC and PR-AUC. Among them, XGBoost stands out by achieving the highest ROC-AUC along with the lowest log loss and Brier score, indicating both superior borrower ranking and the most reliable probability estimates. While CatBoost and LightGBM show competitive PR-AUC and are effective at flagging risky borrowers, their higher logloss and Brier scores suggest less stable calibration. This balance of strong ranking performance and dependable probability estimates makes XGBoost the most suitable choice of tree-based model for prediction.

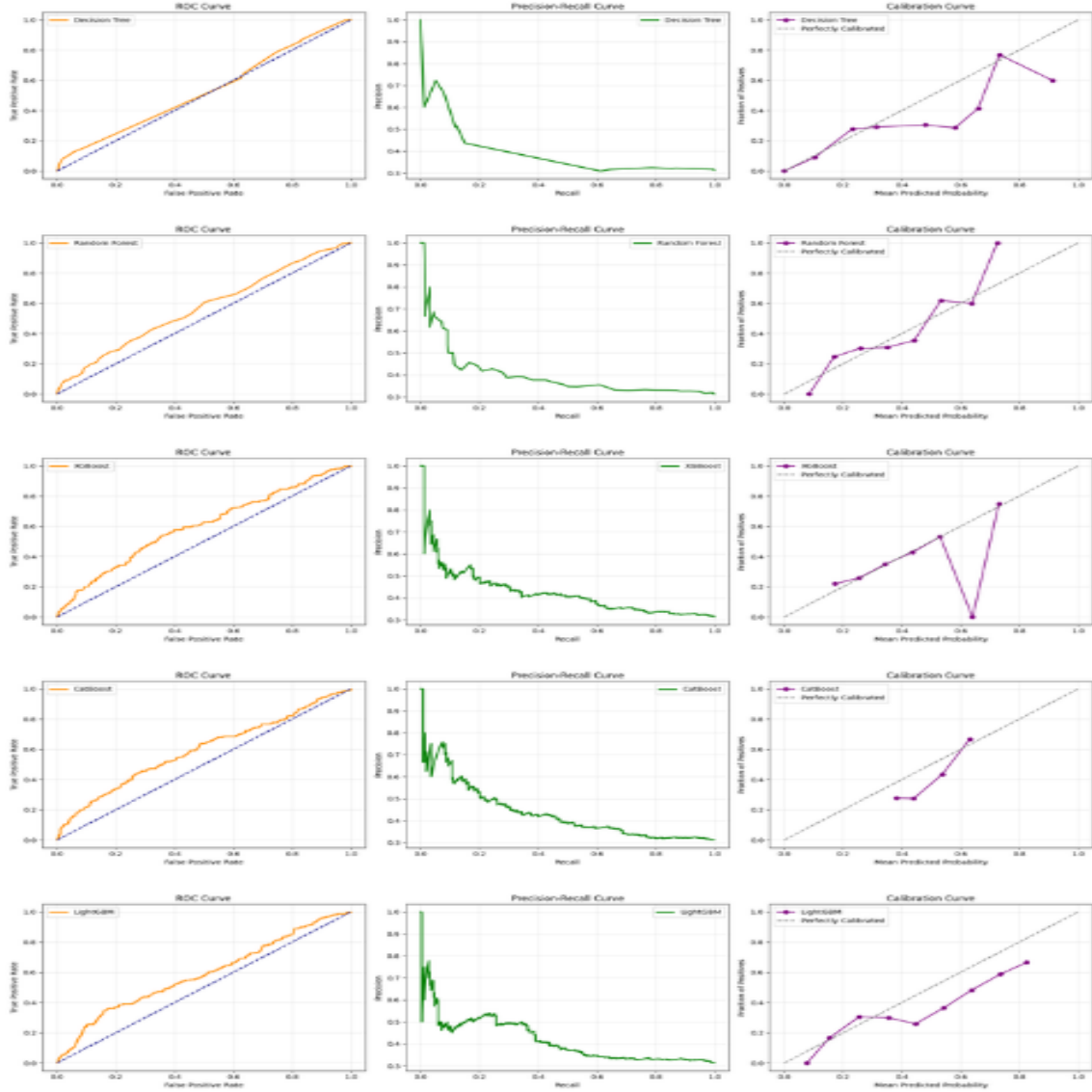


Figure 6: From top to bottom: Decision Trees, Random Forest, XGBoost, Catboost, LightGBM. Graphs of the metrics calculated on the validation set.

The above plots confirm that while the XGBoost model achieves only modest separation power shown by the ROC curve, its probability estimates are surprisingly well-calibrated for the majority of the population. The Calibration Curve demonstrates strong reliability in the 0.2 to 0.55 probability range, indicating that predicted risk scores effectively map to real-world default rates, despite some instability in higher-risk bins. I think it is likely caused by data sparsity in the validation set. Concurrently, the Precision-Recall curve reveals a strict trade-off where high precision is attainable only at minimal recall, suggesting the model is best utilized as a broad, top-of-funnel risk filter rather than a definitive standalone decision-maker.

6 Explainability

I use SHapley Additive exPlanations (SHAP) for to understand the choices made by model and to find out feature importance. I have calculated th SHAP values for all the 5 models and ranked them by their importance by the model.

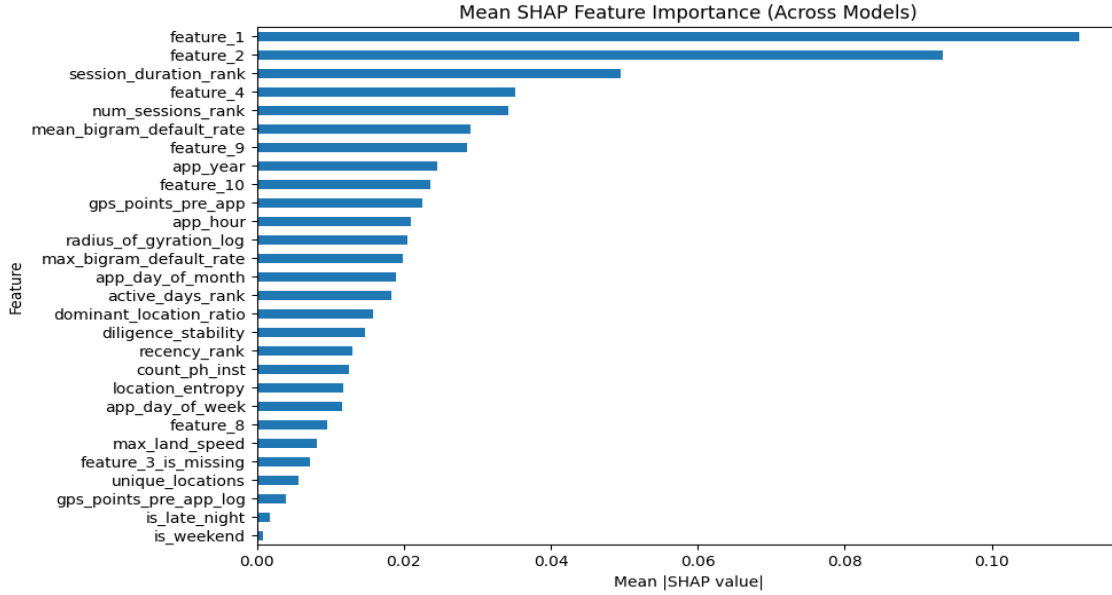


Figure 7: Feature Importance ranked by the Mean SHAP values from the models.

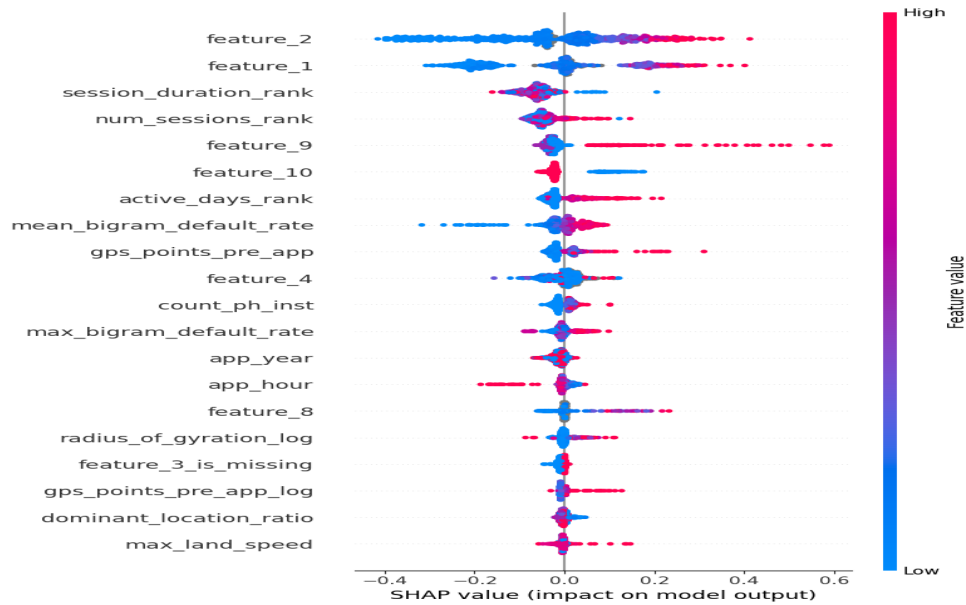


Figure 8: Feature importance for XGBoost model.

Here are a few interpretation made by me from the analysis:

- A small number of features dominate model behavior, which is what we want to see, the model is not overly reliant on a long tail of weak signals.
- The top features show consistent importance across boosted models, suggesting the model is learning stable patterns.
- Behavioral features contribute meaningfully and in the expected direction, indicating that behavioral consistency is a reliable signal for repayment risk.
- GPS features add incremental signal but do not dominate the prediction, which reduces the risk of overfitting to noisy GPS data.
- Features with high missingness or weak signal appropriately contribute little to the model, showing that the model is robust to sparse inputs.

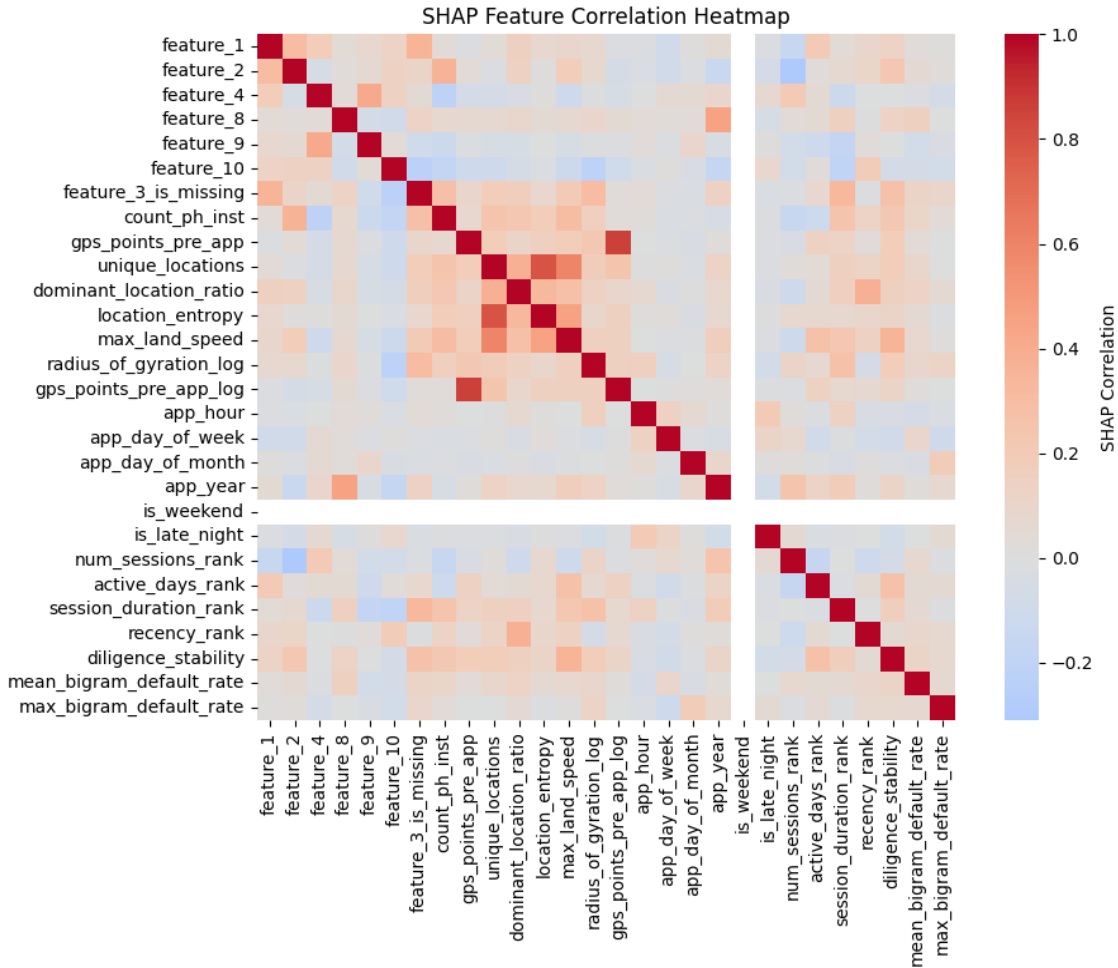


Figure 9: Correlation heatmap between the features from the SHAP values.

So, I understand that the SHAP analysis indicates that the models rely on a small set of stable, intuitive signals and uses secondary features conservatively, which is what we expect from a production-ready risk model. In the future I would love to dive deeper into the SHAP plots of each models to find any underlying pattern in data by examining through the unique lens of each model.

7 Deployment

The overall best performing model XGBoost was deployed using FastAPI. More on that in the GitHub repository.

8 AI Assistance

AI assistance was great when it came to writing the code for visualization and took care of the repeating grunt work required to plot figures and graphs. Apart from that it did collect pretty good reading resources which allowed to gain simple yet effective domain knowledge in modeling the GPS data like common practices and workarounds. Overall I would say the "assistance" aspect of AI allowed me to build up knowledge and sped up my work my quite a lot.

References

- [1] Analytics Vidhya. Do tree-based models outperform deep learning models on tabular data? https://www.analyticsvidhya.com/blog/2022/08/do-tree-based-models-outperform-deep-learning-models-on-tabular-data/?utm_source=chatgpt.com, 2022. Accessed: 2025-12-21.
- [2] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on tabular data?, 2022.
- [3] Anant Nawalgaria, Gabriela Hernandez Larios, Elia Secchi, Mike Styer, Christos Aniftos, and Onofrio Petragallo. Operationalizing generative ai on vertex ai using mlops. <https://www.kaggle.com/whitepaper-operationalizing-generative-ai-on-vertex-ai-using-mlops>, September 2024. Whitepaper.
- [4] Pratham Rana, Kalpana, Chahat Chahat, Soham Modi, Anup Yadav, and Sanjay Singla. Comparative analysis of tree-based models and deep learning architectures for tabular data: Performance disparities and underlying factors. pages 224–231, 12 2023.